安全な障害注入と解除方法

演習専用リソースで、対象、影響、期待結果、中止条件、解除方法を先に確認します。

方法	模擬する状態	解除・確認
Primaryアプリ停止	オリジンが応答できない	再起動し正常性を確認
/health のみ異常	正常性判定により候補から外れる	200応答へ戻す。業務は動く場合あり
障害モード有効化	準備したエラーや応答不能	元の設定へ戻し対象パスを再試験
Front Door経路の拒否	特定経路から接続できない	変更前の規則へ戻す

リソース削除、データ破壊、本番への障害注入は行いません。リージョン障害の完全再現でもありません。

障害発生・検知・最初の成功・安定成功

切り替え時間と利用者の失敗区間を分け、同じ時刻基準と観測条件で記録します。

時刻	記録する事象	評価すること
t0	障害注入が有効になった	測定の起点
t1	Primaryプローブ異常を観測した	観測上の検知。未取得なら未取得と記録
t2	最初にSecondaryの成功を確認した	t2 − t0：最初の成功までの時間
t3	定義した回数・期間で成功が継続した	t3 − t0：演習での安定したWeb応答

アクセス間隔・観測場所・タイムゾーンと、失敗・タイムアウト・遅延の区間を残します。

切り替わらない場合の確認順序

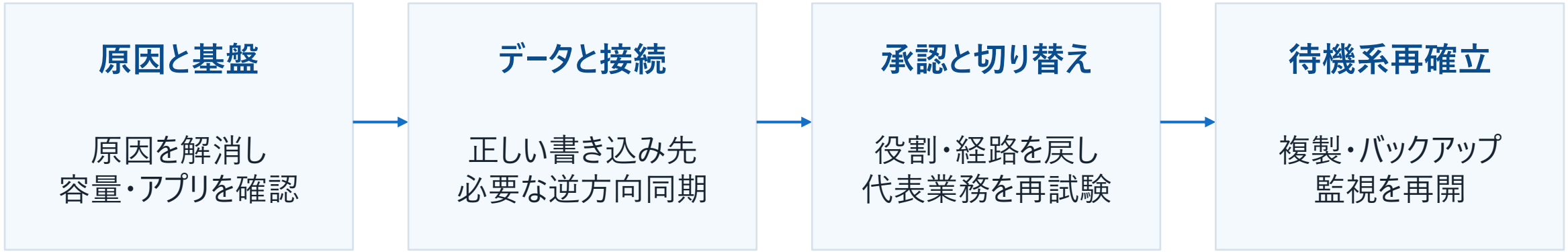
直接アクセス、プローブ、Front Door経由の応答を比較し、期待値との差から調査します。

観測結果	次に確認する場所
Secondaryへの直接アクセスも失敗	アプリ、依存先、容量、認証
直接成功だがプローブは失敗	HEAD／GET、パス、200応答、TLS、アクセス制限
両系正常だが期待先と異なる	ルート、優先度、有効状態、キャッシュ、セッション
Web成功だが業務は失敗	DBの役割、接続先、RBAC、通信経路

変更した項目、根拠、再試験結果を記録し、未確認の原因を断定しません。

データを含むフェールバックの順序

Secondary稼働中に増えたデータと書き込み先を確認してから、承認された手順で戻します。[1]



Primaryが起動しただけで戻さず、二重書き込みや承認前の自動復帰を避ける制御を設計します。

[1] https://learn.microsoft.com/ja-jp/azure/well-architected/reliability/disaster-recovery

DRランブックと定期テスト

発動条件、担当者、権限、操作、期待結果、中止条件と通知を、テスト済みの手順にまとめます。[1][2]

ランブックに記録すること

・誰が発動を判断し、誰が承認・実行するか。

・どの対象へ、どの権限で、何を操作するか。

・成功・中止・切り戻しの条件と
連絡先は何か。

テストで確認すること

・切り替えと業務再開、データ整合性を確認する。

・フェールバックと待機系再確立まで実施する。

・定期・変更後に再試験し、担当者と改善期限を付ける。

承認待ちでRTOを超えた場合は、技術だけでなく判断・承認の運用も改善します。

[1] https://learn.microsoft.com/ja-jp/azure/well-architected/reliability/disaster-recovery

[2] https://learn.microsoft.com/ja-jp/azure/well-architected/reliability/testing-strategy